

Code-Review tools::Field – Befundtabelle (Abschnitt 3)

toolsfield.h · C++20 · Windows/clang 21 · 18 Befunde, sortiert nach Schweregrad und Auftreten im Code

ID	Schw.	Datei/Stelle	Kategorie	Befund	Auswirkung	Empfehlung
F01	Kritisch	Konstruktor Field(bool) / Field(std::string const&)	Overload-Resolution	Field(const char*) fehlt. Bei Field f("text") konkurrieren <code>Field(std::string const&)</code> (benutzerdef. Konversion) und <code>Field(bool)</code> (Standardkonversion Zeiger→bool). Standardkonversionen gewinnen immer.	Jede Konstruktion aus <code>String-Literal/const char*</code> erzeugt still ein <code>Bool-Feld (true)</code> ; der <code>String</code> geht komplett verloren. Kein Compilerfehler.	Explizite Überladung <code>explicit Field(char const * value)</code> ergänzen, die an <code>Field(std::string const&)</code> delegiert.
F02	Kritisch	<code>getInt()/getUInt()/getInt64()/getUInt64()</code> – Zweige <code>Float/Double/LongDouble</code>	Undefiniertes Verhalten	Grenzwertprüfung <code>castet numeric_limits<int>::max()</code> etc. nach <code>float/double/long double</code> . Diese Typen können den Grenzwert oft nicht exakt darstellen und runden aufwärts (z. B. <code>INT_MAX→2147483648.0f</code>).	Werte knapp oberhalb der echten Grenze bestehen die Prüfung; der folgende <code>static_cast<int>(...)</code> liegt dann außerhalb des Zielbereichs → UB. Auf Windows (<code>long double==double</code>) betrifft dies auch <code>getInt64()/getUInt64()</code> .	Vergleich mit exakt darstellbarem Grenzwert (z. B. Zweierpotenz als exklusive Grenze) statt gerundetem Cast durchführen.
F03	Kritisch	<code>getFloat()</code> , <code>getDouble()</code> – Zweige <code>Int/UInt</code>	Undefiniertes Verhalten	<code>static_cast<int>(numeric_limits<float>::lowest())</code> bzw. <code>::max()</code> (analog <code>double</code>) wird berechnet. Diese Werte ($\pm 3.4 \cdot 10^{38}$) liegen weit außerhalb des <code>int</code> -Wertebereichs.	Cast eines Fließkommawerts außerhalb des Zielbereichs in einen <code>Integer</code> ist laut Standard UB – bei jedem Aufruf dieser Zweige, unabhängig vom eigentlichen Wert.	Prüfung entfernen: <code>int/unsigned int</code> liegen wertebereichsmäßig immer innerhalb dessen, was <code>float/double</code> darstellen können. Direkt <code>static_cast<float>(m_intValue)</code> verwenden.
F04	Hoch	Alle expliziten Wert-Konstruktor	Overload-Resolution / Compilerfehler	Es fehlen Überladungen für <code>long</code> , <code>unsigned long</code> , <code>short</code> , <code>unsigned short</code> , <code>char</code> , <code>wchar_t</code> usw. Mehrere Konversionen sind dafür gleichrangig.	Übergabe eines <code>long/unsigned long</code> -Werts (z. B. viele <code>WinAPI</code> -Typen wie <code>DWORD</code>) führt zu einem Compilerfehler wegen Mehrdeutigkeit.	Fehlende Überladungen ergänzen (mind. <code>long</code> , <code>unsigned long</code>) oder bewusst <code>per =delete</code> sperren für klare Fehlermeldung.
F05	Hoch	<code>getBool()</code> vs. übrige <code>get*()</code> – Zweig <code>DateTime</code>	Semantischer Widerspruch	Numerische Getter interpretieren <code>DateTime</code> als "Sekunden seit Mitternacht" (Datum verworfen). <code>getBool()</code> prüft stattdessen <code>isValid()</code> – ein anderes Kriterium.	Zwei undokumentierte, inkonsistente Konventionen für denselben Quelltyp innerhalb derselben Klasse.	Konvention vereinheitlichen und dokumentieren, z. B. <code>getBool()</code> auf <code>secondsOfDay() != 0</code> umstellen.
F06	Mittel	Datei-Kopfkommentar vs. <code>m_isNull/isNull()</code>	Widersprüchliche Dokumentation	Kommentar: auf <code>isNull</code> werde verzichtet (lösbar über <code>std::optional</code>). Tatsächlich implementiert die Klasse <code>isNull()</code> , <code>m_isNull</code> und einen "Null"-Default-Konstruktor.	Irreführende Dokumentation für zukünftige Bearbeiter.	Kommentar an die tatsächliche Implementierung anpassen.
F07	Mittel	<code>Field::Field()</code> (Default-Ctor), <code>m_type</code> -Default	Fehlende/verletzte Invariante	Ein "Null"-Feld hat wegen des Default-Member-Initializers immer <code>getType() == DataType::String</code> , unabhängig vom gewünschten Typ. Kein typisiertes <code>NULL</code> möglich.	Code, der anhand <code>getType()</code> verzweigt, kann ein <code>NULL-Feld</code> fälschlich als "leerer String" statt "kein Wert" behandeln.	<code>DataType::Null</code> ergänzen und im Default-Ctor setzen, oder <code>Field(DataType)</code> für typisierte <code>NULLs</code> anbieten.
F08	Mittel	Alle <code>get*()</code> -Methoden	Erweiterbarkeit	Statt <code>switch(m_type)</code> wird durchgängig eine <code>if</code> -Kette verwendet. Der Compiler kann bei einer <code>if</code> -Kette nicht wie bei <code>switch (-Wswitch)</code> vor fehlenden Fällen warnen.	Wird <code>DataType</code> künftig erweitert (z. B. <code>Blob</code>), fällt jede Methode ohne Warnung still auf den letzten Rückgabewert zurück.	Auf <code>switch(m_type)</code> ohne default-Zweig umstellen, damit der Compiler fehlende Fälle meldet.

ID	Schw.	Datei/Stelle	Kategorie	Befund	Auswirkung	Empfehlung
F09	Mittel	getInt()/getUInt()/getFloat()/getDouble()/getBool() vs. getInt64()/getUInt64()/getLongDouble()	Inkonsistenz	Zwei String-Parsing-Mechanismen: tools::string_to_* vs. direktes std::stoll/std::stoull/std::stold (wirft invalid_argument/out_of_range).	Ungültige String-Werte führen je nach Getter zu unterschiedlichem Verhalten (still 0 vs. Exception) – schwer vorhersehbar.	Auf einen einheitlichen Parsing-Mechanismus vereinheitlichen.
F10	Mittel	Private Member-Liste (alle m_*Value-Member)	Design / Speicher-Overhead	Für jeden der zehn Typen existiert ein eigenes Member gleichzeitig (kein std::variant/union). Jede Instanz trägt die Summe aller Typgrößen.	Unnötig hoher Speicherverbrauch und Konstruktionskosten; widerspricht dem Kommentar "Field ist ein eigener Variant".	Umbau auf std::variant erwägen (Objektgröße auf Größe des größten Typs reduzieren, Invariante Typ↔Wert wird vom Compiler garantiert).
F11	Mittel	getUInt(), getInt64(), getUInt64(), getFloat(), getDouble() – negative-Wert-Prüfungen	Semantisch unpassende Exception	std::underflow_error wird für negative signed-Werte verwendet, die nicht in unsigned passen. Laut Standard ist underflow_error für Fließkomma-Unterlauf gedacht.	Aufrufer, die anhand des Exception-Typs unterscheiden, erhalten eine irreführende Klassifikation.	std::out_of_range oder eine eigene, aussagekräftige Exception-Klasse verwenden.
F12	Mittel	Field(Field const&)=delete; operator=(Field const&)=delete;	Design / Nutzbarkeit	Field ist ein reiner Move-Only-Typ. Für einen kleinen, unveränderlichen Wertetyp (typischerweise in Zeilen/Ergebnismengen) ungewöhnlich.	std::vector<Field> row; row.push_back(f); mit benanntem (lvalue) f kompiliert nicht mehr.	Falls Kopierbarkeit gewünscht: Copy-Ctor/-Assignment implementieren bzw. =default. Sonst kurz dokumentieren, warum Move-Only.
F13	Mittel	getDateTime()	Unvollständige Konvertierung	Für alle Nicht-DateTime-Typen wird ausnahmslos geworfen – auch für String, obwohl sonst überall String→Zahl/Bool konvertiert wird.	Ein Feld mit z. B. ISO-8601-String kann nicht als DateTime gelesen werden, obwohl fachlich naheliegend.	Klären, ob String→DateTime-Konvertierung gewünscht ist; ggf. über vorhandene Parsing-Funktion ergänzen.
F14	Niedrig	Field::get()	Namensgebung	get() liefert identisch zu getString() immer einen String. Der generische Name suggeriert typunabhängigen bzw. nativen Zugriff.	Verwirrende öffentliche Schnittstelle; Aufrufer könnten anderen Rückgabetyt erwarten.	Methode umbenennen/entfernen (redundant zu getString()) oder Zweck im Namen klarstellen.
F15	Niedrig	getDateTime()	DRY-Verstoß	Zehn nahezu identische if-Blöcke, die sich nur in der Fehlermeldung unterscheiden.	Hoher Wartungsaufwand bei Änderungen; Copy-Paste-Risiko.	Auf switch mit gemeinsamer, typnamen-parametrisierter Fehlermeldung reduzieren.
F16	Hinweis	getString() – Zweige Float/Double/LongDouble	Formatierungsqualität	std::to_string() liefert für Fließkommatypen eine feste Anzahl von sechs Nachkommastellen (z. B. "3.140000").	Evtl. nicht das gewünschte Format für Anzeige-/Exportzwecke.	Bei Bedarf std::format (C++20) mit definierter Präzision verwenden.
F17	Hinweis	throwIfNull()	Diagnosequalität	Fehlermeldung ("Field is null") enthält keinen Kontext, welcher Getter aufgerufen wurde.	Erschwerte Fehlersuche bei mehreren get*()-Aufrufen in einem größeren Ausdruck.	Aufrufenden Methodennamen (z. B. via std::source_location) in die Meldung aufnehmen.
F18	Hinweis	Alle get*()	Optionale Verbesserung	Kein [[nodiscard]] an den Gettern einer reinen Wertklasse.	Versehentliches Verwerfen eines Rückgabewerts wird nicht vom Compiler erkannt.	[[nodiscard]] ergänzen.